Advanced Rust (2026): Performance, Profiling, Measurement

Extension Lecture

Lukáš Hozda

Spring 2026

MFF CUNI

Measurement

Performance Claims

- A performance claim without workload, method, and environment is anecdote

Performance Claims

- A performance claim without workload, method, and environment is anecdote
- Throughput, latency, memory use, and tail behavior are different metrics

Performance Claims

- A performance claim without workload, method, and environment is anecdote
- Throughput, latency, memory use, and tail behavior are different metrics
- Optimization begins with a question, not with code motion

What Must Be Reported

- Input shape and size

What Must Be Reported

- Input shape and size
- Rust version, target, and build mode

What Must Be Reported

- Input shape and size
- Rust version, target, and build mode
- Machine characteristics that matter

What Must Be Reported

- Input shape and size
- Rust version, target, and build mode
- Machine characteristics that matter
- Number of runs and observed variance

Microbenchmark Pitfall

```rust
use std::time::Instant;

let start = Instant::now();
let result = compute();
let elapsed = start.elapsed();
println!("{result} in {elapsed:?}");
```

- This is often too noisy to tell you anything useful

Microbenchmark Pitfall

```rust
use std::time::Instant;

let start = Instant::now();
let result = compute();
let elapsed = start.elapsed();
println!("{result} in {elapsed:?}");
```

- This is often too noisy to tell you anything useful
- Single-run timing is almost never enough

Release Mode

- Benchmarking debug builds is usually pointless

Release Mode

- Benchmarking debug builds is usually pointless
- Optimizer changes both absolute speed and code shape

Release Mode

- Benchmarking debug builds is usually pointless
- Optimizer changes both absolute speed and code shape
- Build mode is part of the experiment

black_box

```rust
use std::hint::black_box;

let x = black_box(20_u64);
let y = fibonacci(x);
black_box(y);
```

- `black_box` helps stop the optimizer from deleting the benchmarked work

black_box

```rust
use std::hint::black_box;

let x = black_box(20_u64);
let y = fibonacci(x);
black_box(y);
```

- `black_box` helps stop the optimizer from deleting the benchmarked work
- It does not magically make the benchmark realistic

Criterion

```rust
use criterion::{black_box, Criterion};

fn bench_sum(c: &mut Criterion) {
    let xs: Vec<i64> = (0..100_000).collect();
    c.bench_function("sum", |b| {
        b.iter(|| xs.iter().copied().sum::<i64>())
    });
}
```

- Criterion handles warmup, repeated sampling, and statistics for you

Benchmark Layers

- Microbenchmarks isolate local effects

Benchmark Layers

- Microbenchmarks isolate local effects
- End-to-end benchmarks show interaction costs and queueing effects

Benchmark Layers

- Microbenchmarks isolate local effects
- End-to-end benchmarks show interaction costs and queueing effects
- You usually need both

Profiling

Sampling Profilers

- Profilers answer where time is actually going

Sampling Profilers

- Profilers answer where time is actually going
- This often contradicts intuition

Sampling Profilers

- Profilers answer where time is actually going
- This often contradicts intuition
- Sampling is usually the first tool, not the last

Typical Tools

- `perf` on Linux

Typical Tools

- `perf` on Linux
- `cargo flamegraph` as a convenient wrapper

Typical Tools

- `perf` on Linux
- `cargo flamegraph` as a convenient wrapper
- `cargo asm` or Compiler Explorer to inspect generated code

Flamegraph

- Width corresponds to sampled time

Flamegraph

- Width corresponds to sampled time
- Deep stack is not itself a problem

Flamegraph

- Width corresponds to sampled time
- Deep stack is not itself a problem
- Wide unexpected frames are what you look for

Hardware Counters

- Cycles, instructions, cache misses, branch misses

Hardware Counters

- Cycles, instructions, cache misses, branch misses
- Counter values are only meaningful in context of the workload

Hardware Counters

- Cycles, instructions, cache misses, branch misses
- Counter values are only meaningful in context of the workload
- Use them to guide hypotheses, not as standalone truth

CPU and Memory Effects

Cache Locality

- Contiguous access patterns often beat clever pointer-heavy structures

Cache Locality

- Contiguous access patterns often beat clever pointer-heavy structures
- Big-O notation says nothing about cache misses

Cache Locality

- Contiguous access patterns often beat clever pointer-heavy structures
- Big-O notation says nothing about cache misses
- Data layout is part of algorithm design

Allocation Pressure

- Repeated allocation can dominate hot paths

Allocation Pressure

- Repeated allocation can dominate hot paths
- Reusing buffers and reserving capacity can matter more than arithmetic tweaks

Allocation Pressure

- Repeated allocation can dominate hot paths
- Reusing buffers and reserving capacity can matter more than arithmetic tweaks
- Measure allocator behavior before rewriting everything around arenas

Branch Prediction

- Predictable branches are cheap

Branch Prediction

- Predictable branches are cheap
- Highly data-dependent branches can stall pipelines

Branch Prediction

- Predictable branches are cheap
- Highly data-dependent branches can stall pipelines
- Table layout or data transformation can beat branch micro-tuning

False Sharing

```
use std::sync::atomic::AtomicU64;

#[repr(align(64))]
struct PaddedCounter(AtomicU64);
```

- Independent variables on the same cache line can still fight each other

False Sharing

```
use std::sync::atomic::AtomicU64;

#[repr(align(64))]
struct PaddedCounter(AtomicU64);
```

- Independent variables on the same cache line can still fight each other
- This is a hardware effect, not a logical data-race issue

Concurrency Performance

Lock Contention

- Lock-free is not automatically faster

Lock Contention

- Lock-free is not automatically faster
- Contention frequency and critical-section length matter more than slogans

Lock Contention

- Lock-free is not automatically faster
- Contention frequency and critical-section length matter more than slogans
- Sharding often beats global locking and also beats overcomplicated lock-free code

Sharded Counter Shape

```
use std::sync::atomic::{AtomicU64, Ordering};

struct Sharded {
    shards: Vec<AtomicU64>,
}

impl Sharded {
    fn inc(&self, shard: usize) {
        self.shards[shard].fetch_add(1, Ordering::Relaxed);
    }
}
```

Queue Bounds

- Unbounded queues can convert bursty load into memory blow-up

Queue Bounds

- Unbounded queues can convert bursty load into memory blow-up
- Bounded queues impose backpressure explicitly

Queue Bounds

- Unbounded queues can convert bursty load into memory blow-up
- Bounded queues impose backpressure explicitly
- Queue policy is part of system correctness, not just throughput tuning

Static vs Dynamic Work Distribution

- Static chunking has low scheduling overhead

Static vs Dynamic Work Distribution

- Static chunking has low scheduling overhead
- Dynamic stealing handles imbalance better

Static vs Dynamic Work Distribution

- Static chunking has low scheduling overhead
- Dynamic stealing handles imbalance better
- Which one wins depends on task variance

Instrumentation

Metrics

- Benchmarks tell you what happens in controlled runs

Metrics

- Benchmarks tell you what happens in controlled runs
- Metrics tell you what happens in long-running real systems

Metrics

- Benchmarks tell you what happens in controlled runs
- Metrics tell you what happens in long-running real systems
- These are different tools for different questions

Prometheus Counter

```rust
use prometheus::{Counter, Opts, Registry};

let counter_opts = Opts::new("my_counter", "A counter for tracking events");
let counter = Counter::with_opts(counter_opts).unwrap();
let registry = Registry::new();
registry.register(Box::new(counter.clone())).unwrap();
counter.inc();
```

Prometheus Gauge

```rust
use prometheus::{Gauge, Opts, Registry};

let gauge_opts = Opts::new("my_gauge", "A gauge for tracking a value");
let gauge = Gauge::with_opts(gauge_opts).unwrap();
let registry = Registry::new();
registry.register(Box::new(gauge.clone())).unwrap();
gauge.set(10.0);
```

Histograms and Labels

- Histograms are the usual tool for latency distributions

Histograms and Labels

- Histograms are the usual tool for latency distributions
- Labels are useful and dangerous

Histograms and Labels

- Histograms are the usual tool for latency distributions
- Labels are useful and dangerous
- High-cardinality labels can destroy your metrics backend

Instrumentation Discipline

- Instrument critical paths, not everything that moves

Instrumentation Discipline

- Instrument critical paths, not everything that moves
- Avoid emitting metrics whose meaning is unclear

Instrumentation Discipline

- Instrument critical paths, not everything that moves
- Avoid emitting metrics whose meaning is unclear
- A cheap, stable metric is better than an expensive decorative one

Reporting

Reproducibility

- Commit benchmark code

Reproducibility

- Commit benchmark code
- Keep inputs or generators under version control

Reproducibility

- Commit benchmark code
- Keep inputs or generators under version control
- Write down command lines and environment assumptions

Reproducibility

- Commit benchmark code
- Keep inputs or generators under version control
- Write down command lines and environment assumptions
- Otherwise nobody can tell whether the speedup is real

Comparison Table

- Baseline

Comparison Table

- Baseline
- Changed version

Comparison Table

- Baseline
- Changed version
- Absolute numbers

Comparison Table

- Baseline
- Changed version
- Absolute numbers
- Relative change

Comparison Table

- Baseline
- Changed version
- Absolute numbers
- Relative change
- Confidence or variance estimate

Common Failure Modes

- Only one run

Common Failure Modes

- Only one run
- No warmup

Common Failure Modes

- Only one run
- No warmup
- Debug build

Common Failure Modes

- Only one run
- No warmup
- Debug build
- No correctness check after optimization

Common Failure Modes

- Only one run
- No warmup
- Debug build
- No correctness check after optimization
- Benchmark optimized away

Summary

- Measure before and after, not only after

Summary

- Measure before and after, not only after
- Profile before guessing

Summary

- Measure before and after, not only after
- Profile before guessing
- Treat cache, allocation, contention, and queueing as first-class effects

Summary

- Measure before and after, not only after
- Profile before guessing
- Treat cache, allocation, contention, and queueing as first-class effects
- Production metrics and offline benchmarks complement each other

Questions?